



分布与并行数据库

7 分布式恢复管理

主讲教师：鄂金龙
中国人民大学信息学院



主要内容

- 分布式恢复概述
- 集中式数据库的故障恢复
- 分布式事务的故障恢复
- 分布式可靠性协议
- 大数据数据库的恢复管理

分布式恢复概述

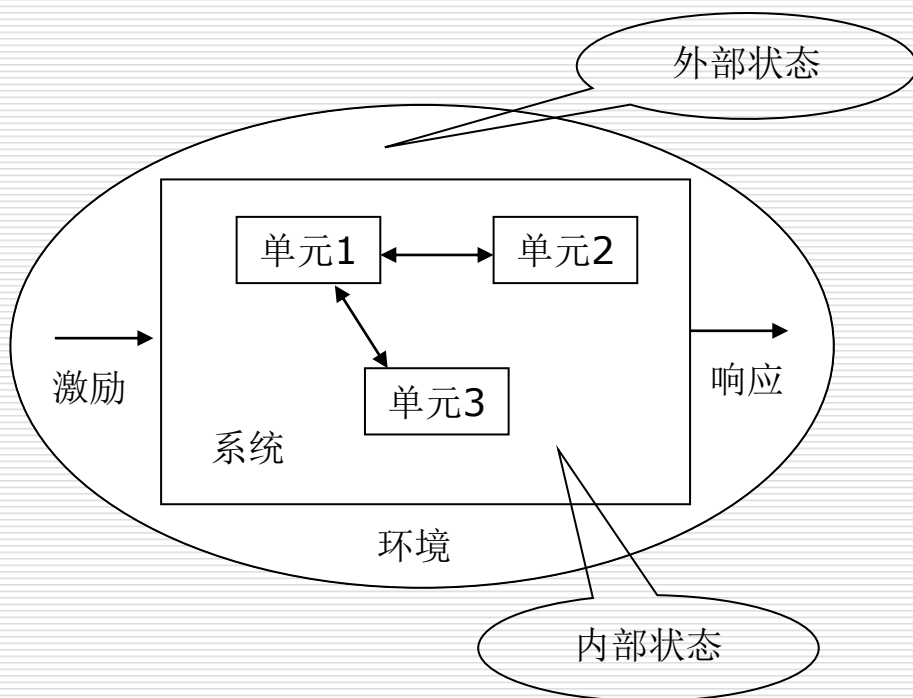
数据库系统的可恢复性和高可靠性是保证各种应用正确而可靠地运行所不可缺少的重要组成部分。

- ▶ 可恢复性，提供恢复措施，把数据库系统从故障状态恢复到一个已知的正确状态；
- ▶ 高可靠性，尽量将崩溃后数据库的不可用的时间减少到最低，并保证事务的原子性和耐久性。

分布式恢复概述

□ 故障模型

系统的组成



故障的三种形式

- ▶ 故障 (Fault)
- ▶ 错误 (Error)
- ▶ 失效 (Failure)

分布式恢复概述

□ 故障模型

故障（Fault）：系统单元内部发生的错误或系统内部设计错误；

错误（Error）：系统单元内出现了不正确的状态，是故障的内在表现形式；

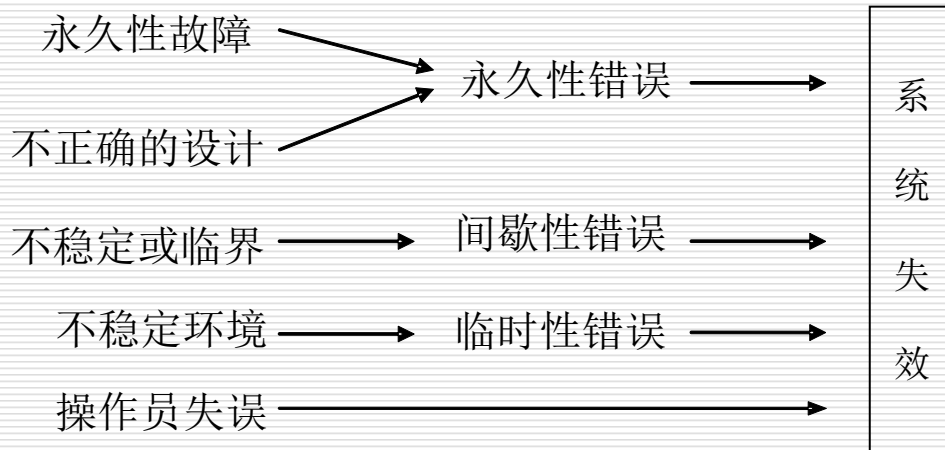
失效（Failure）：指系统的外部状态中所表现出来的错误；



当系统单元组建不合理或系统内部设计存在不足时，将会引发系统故障，此时系统的内部状态处于错误的状态，进而使系统的外部环境受到影响，最终产生失效。

分布式恢复概述

故障模型



永久性错误：会一直持续下去，不会自动恢复，如硬盘损坏等。

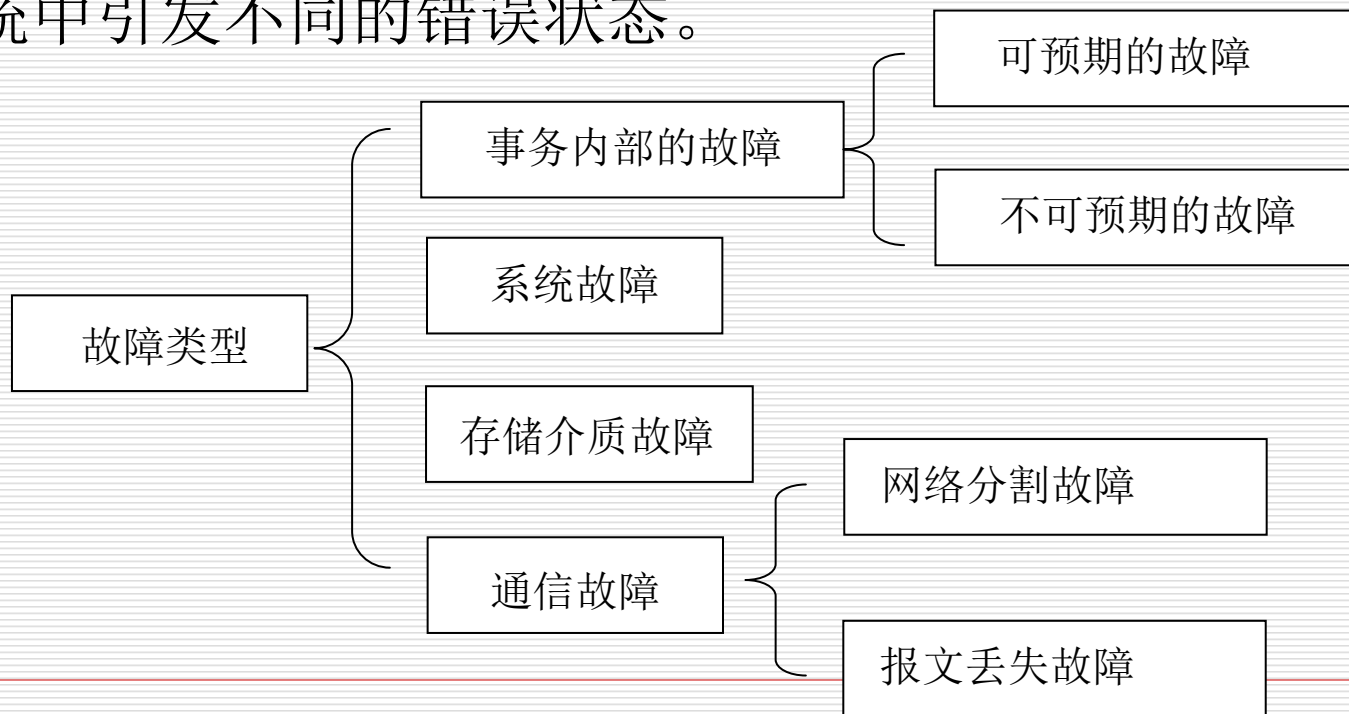
间歇性错误：时常发生，会自动恢复，如电路接触故障、计算机运行不稳定等。

暂时性错误：可能发生一次便不再发生，如偶然电流干扰导致的瞬间读写错误。

分布式恢复概述

□ 故障模型

数据库系统中可能发生各种各样的故障，每种故障都会在系统中引发不同的错误状态。



分布式恢复概述

□ 故障模型

数据库系统中的故障通常分为4类：

（1）事务内部的故障

事务内部的故障可细分为可预期的和不可预期的。

可预期的事务故障是指故障的发生可以通过事务程序本身来检测。例如：在转帐事务中，事务主动检查贷方余额是否足够支持转帐，若发现不足，则废弃该子事务，数据库状态没有因为此事务发生变化；

不可预期的事务故障是指故障的发生不能被应用程序所检测并处理。例如：死锁、算术溢出、完整性被破坏、操作员失误等故障都是不可预期的事务故障，这类故障无法由事务程序本身所预测。

事务内部的故障大多数都是不可预期的，发生这类故障的事务将不能正常运行到其终点位置（Commit或Abort）。因此，针对这类故障，数据库恢复机制要强行废弃该事务，使数据库回滚到事务执行前的状态。

分布式恢复概述

□ 故障模型

数据库系统中的故障通常分为4类：

(2) 系统故障

- **表现：**使系统停止运转，必须经过重启后系统才能恢复正常。例如，CPU故障、系统死循环、缓冲区溢出、系统断电等。
- **特点：**仅使正在运行的事务受到影响，但数据库本身没有被破坏；**内存中的数据全部丢失**。一方面，一些**尚未完成的事务的结果**可能已被写入数据库中；另一方面，一些**已提交的事务的结果**可能还未更新到磁盘上。
- **结果：**系统故障发生后数据库可能处于**不一致**的状态。

对于系统故障，数据库恢复机制要在系统重启后，将所有非正常终止的事务强行废弃，同时将已提交的事务的结果重新更新到数据库中，以保证数据库的正确性。

分布式恢复概述

□ 故障模型

数据库系统中的故障通常分为4类：

（3）存储介质故障

存储介质故障是指存储数据的磁盘等硬件设备发生的故障。例如，磁盘坏损、磁头碰撞、瞬时强磁场干扰等均为存储介质故障。

特点：不仅使正在运行的**所有事务**受到影响，而且**数据库**本身也被破坏。

结果：同前两种故障相比，存储介质故障是一种较**严重**的故障类型。

对于存储介质故障，数据库恢复机制要定期地对数据库进行**转储**，借助于备份数据库和日志文件来进行故障恢复。

分布式恢复概述

□ 故障模型

数据库系统中的故障通常分为4类：

（4）通信故障

前三个故障都是单场地故障、也是集中式数据库中的故障。对于分布式数据库而言，还要进行各个场地的通信，因此还会产生通信故障。

通信故障可细分为网络分割故障和报文丢失故障。

- ▶ 网络分割是指通讯网络中一部分场地和另一部分场地之间完全失去联系。
- ▶ 报文丢失是指报文本身错误或在传送过程中丢失而导致数据不正确。

对于通信故障，数据库恢复机制要针对故障的表现形式进行判断，分析故障产生原因并进行相应的处理，如重发报文等。

分布式恢复概述

□ 故障模型

事务内部故障

系统故障

通信故障

存储介质故障

软故障—临时性的、
间歇性的，导致软
失效。

硬故障—永久性的、
不能自行恢复、致
命的，导致硬失效。

分布式恢复概述

□ 恢复模型

在故障恢复过程中，数据库恢复管理器依据数据库日志文件（log）对数据库事务进行恢复操作。

- ▶ 日志文件的格式和内容
- ▶ 反做(undo)和重做(redo)恢复策略
- ▶ 故障恢复模型

分布式恢复概述

□ 恢复模型

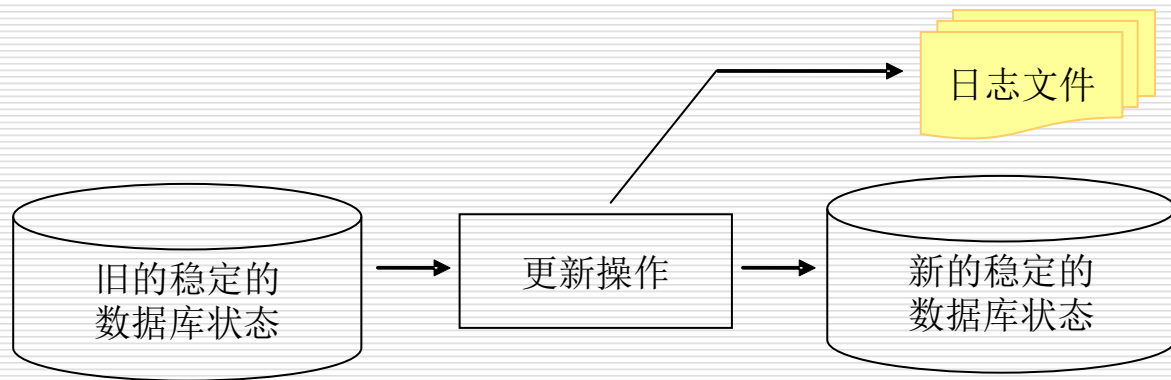
◆ 数据库的日志文件

内容：记录事务的执行信息。

作用：① 用来进行数据库的恢复；

② 数据库使用情况统计和分析的数据源。

有多重要？先写日志原则(WAL, Write Ahead Log)。



分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件

在日志中，系统的运行情况通常以运行记录的形式存放。日志记录可分为：

- ▶ 数据日志记录
- ▶ 命令日志记录
- ▶ 检查点日志记录

分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件

数据日志记录包括：

- ▶ 事务标识符（标明是哪个事务）
- ▶ 操作类型（插入、删除或修改）
- ▶ 操作的数据项
- ▶ 数据项的旧值（前像，BeforeImage）
- ▶ 数据项的新值（后像，AfterImage）

分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件

命令日志记录包括：

- ▶ 事务标识符（标明是哪个事务）；
- ▶ 命令（如：Begin、Abort或Commit）。

分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件

检查点记录

检查点是在日志中周期设定的操作标志，目的是减少系统故障后的恢复的工作量。在检查点上，需要完成的操作包括：

首先，将日志缓冲区中的内容写入外存中的日志；

然后，在外存日志中登记一个检查点记录；

接下来，将数据库缓冲区的内容写入外存数据库；

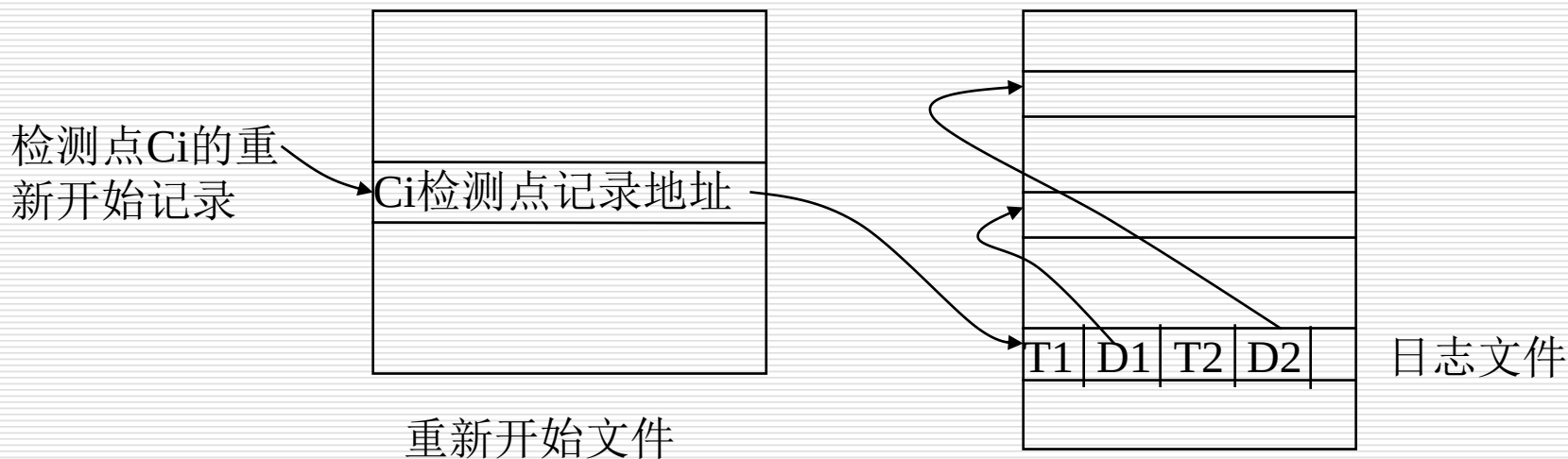
最后，把外存日志中检查点的地址写入重启动文件。

上述工作的目的就是把检查点之前的工作持久化。

分布式恢复概述

□ 检测点记录的内容

- ◆ 建立检测点时刻所有正在执行的事务清单
- ◆ 这些事务最近一个日志记录的地址

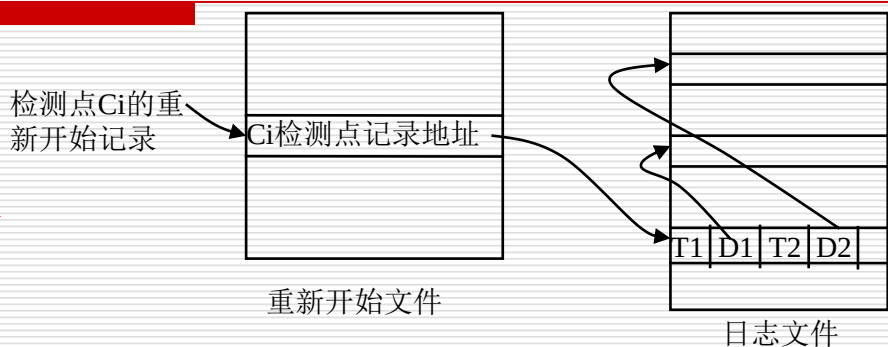


分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件

检查点记录



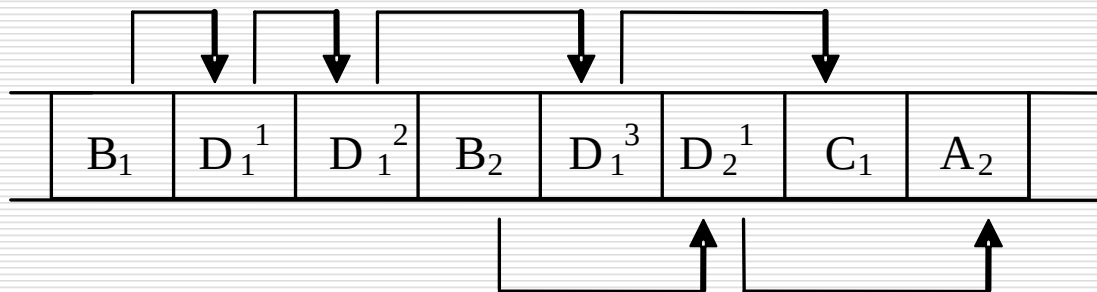
当系统出现故障时，只需要进行如下恢复处理：在**重新启动文件**中找到最近的一次检查点记录在日志中的位置，然后在日志中找到**最近的一个检查点**。对最近的检查点以后的提交操作进行恢复处理；对**最近检查点没提交的活动事务**的操作进行恢复处理；对**检查点以后没有提交的事务**的操作进行恢复处理。

可见，基于检查点的恢复处理可有效减少恢复的工作量（只需要恢复最近一个检查点之后的数据库操作）。

分布式恢复概述

□ 恢复模型

◆ 数据库的日志文件



其中， D_i^j 表示事务 i 对数据 D 的第 j 步操作， B_i 表示事务 i 开始执行(begin)， C_i 和 A_i 分别表示事务 i 的提交(commit)与废弃(abort)。

T_1 的执行过程为： $B_1 \rightarrow D_1^1 \rightarrow D_1^2 \rightarrow D_1^3 \rightarrow C_1$

T_2 的执行过程为： $B_2 \rightarrow D_2^1 \rightarrow A_2$

分布式恢复概述

□ 恢复模型

◆ 反做(undo)和重做(redo)恢复策略

什么是反做？

- 反做（undo）也称撤消、回滚（rollback），是将一个数据项的值恢复到其修改之前的值，即取消一个事务所完成的操作结果。
- 当一个事务尚没提交时，如果缓冲区管理器允许该事务修改过的数据写到外存数据库，一旦此事务出现故障需废弃时，就需对被这个事务修改过的数据项进行反做，即根据日志文件将其恢复到前像。反做的目的是保持数据库的原子性。

反做幂等率： $\text{undo}(\text{undo}(\dots T)) = \text{undo}(T)$

分布式恢复概述

□ 恢复模型

◆ 反做(undo)和重做(redo)恢复策略

什么是重做？

- 重做（redo）也称为前滚操作（rollforward），是将一个数据项的值恢复到其修改后的值，即恢复一个事务的操作结果。
- 当一个事务提交时，如果缓冲区管理器允许该事务修改过的数据不立刻写到外存数据库，一旦此事务出现故障，需对被这个事务修改过的数据项进行重做（redo），即根据日志文件将其恢复到后像。重做（redo）的目的是保持数据库的持久性。

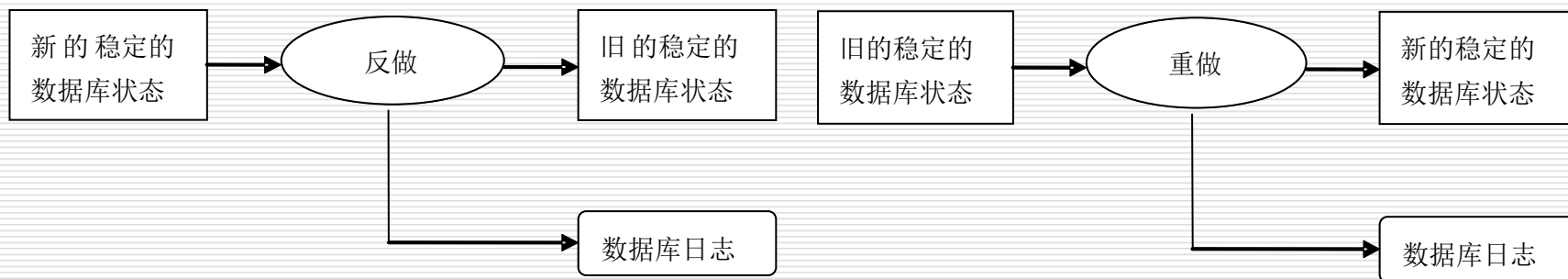
重做幂等率： $\text{redo}(\text{redo}(\dots T)) = \text{redo}(T)$

分布式恢复概述

□ 软/硬故障的恢复模型

◆ 软故障的恢复模型

当**软故障**发生时，造成数据库不一致状态的原因包括：**一**、一些未完成事务对数据库的更新已写入外存数据库；**二**、一些已提交事务对数据库的更新还没来得及写入外存数据库。因此，**需要的恢复操作有undo和redo**。



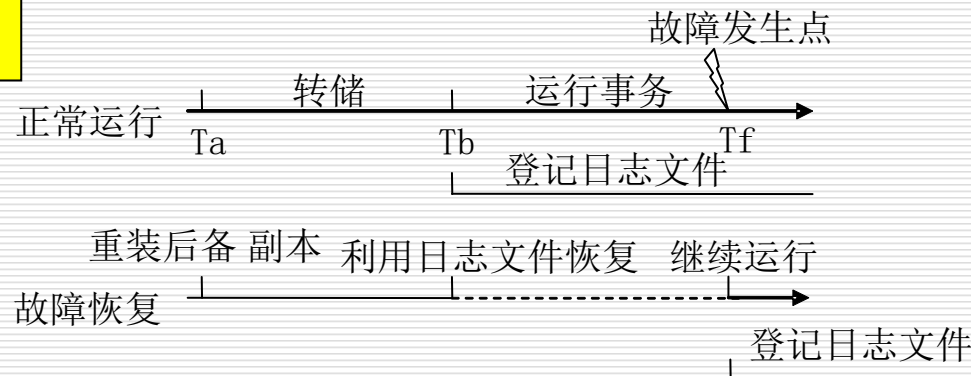
分布式恢复概述

□ 软/硬故障的恢复模型

◆ 硬故障的恢复模型

硬故障的主要恢复措施是进行数据转储和建立日志文件。首先，DBA要定期地将数据库转储到其它磁盘上，形成一系列备份数据库（也称“后备副本”）。接下来，利用日志文件重新运行转储以后的所有更新事务，使数据库再进一步地恢复到故障发生时的状态。

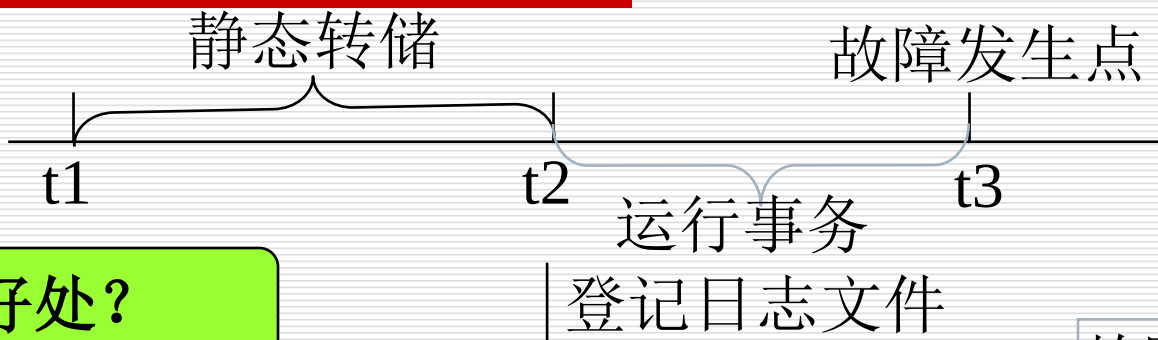
静态转储方式恢复：



分布式恢复概述

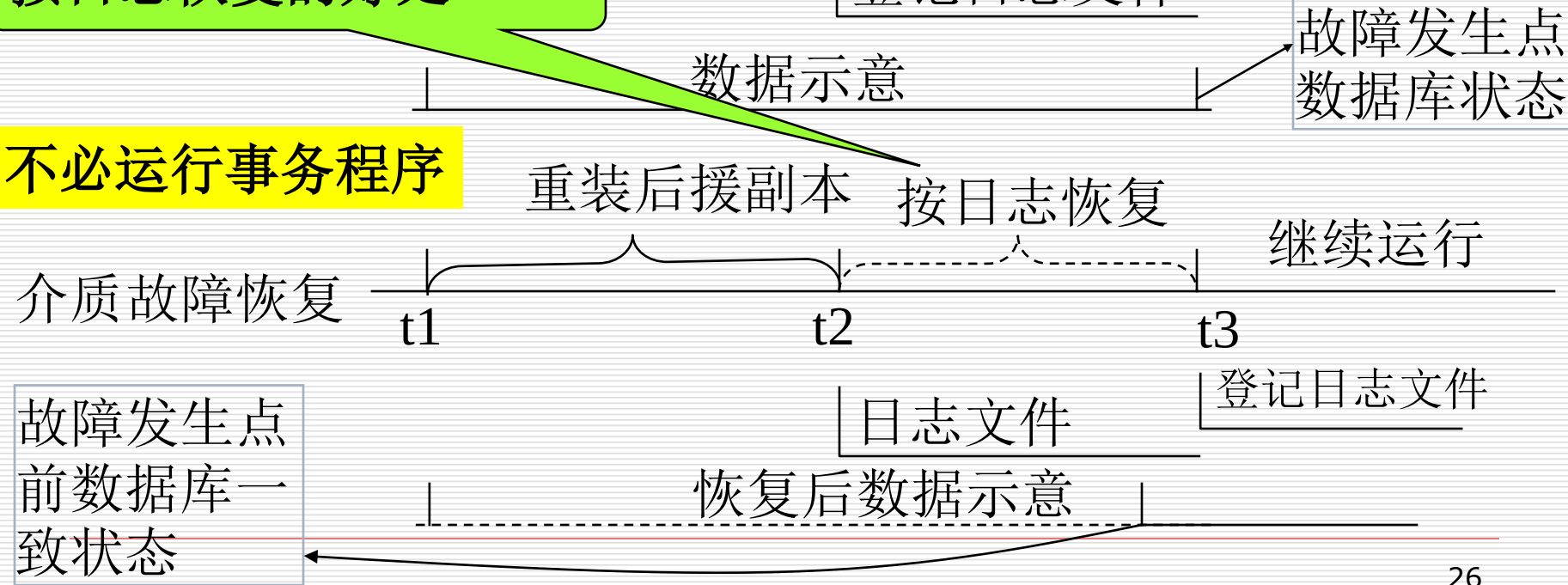
静态转储方式恢复:

正常运行:



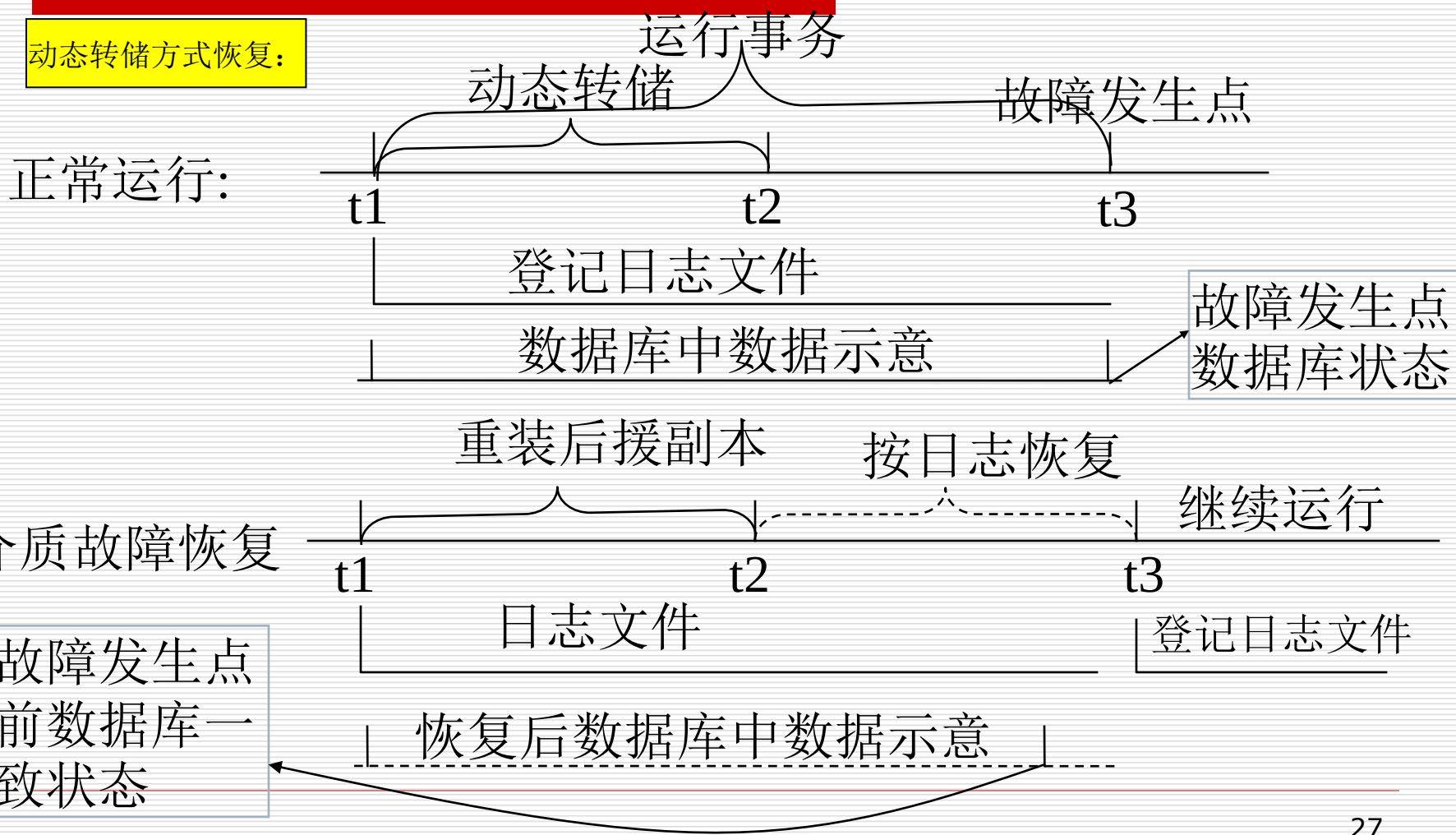
按日志恢复的好处?

不必运行事务程序



分布式恢复概述

动态转储方式恢复:



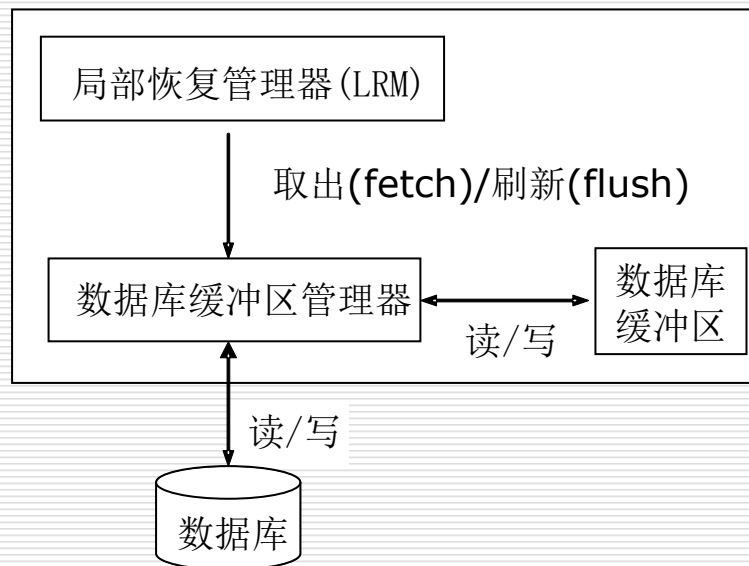
集中式数据库的故障恢复

□ 局部恢复系统的体系结构

数据库存储在永久性的外存设备上。

数据库缓冲区用来存放最近执行的事务所使用的数据。数据库缓冲区被放置在具有挥发性的内存中，以页为单位来缓存数据。

数据库缓冲区管理器负责读写数据库及缓冲区中的数据。局部恢复管理器与缓冲区管理器之间存在两个交互接口：读取数据页（fetch）和刷新数据页（flush）。

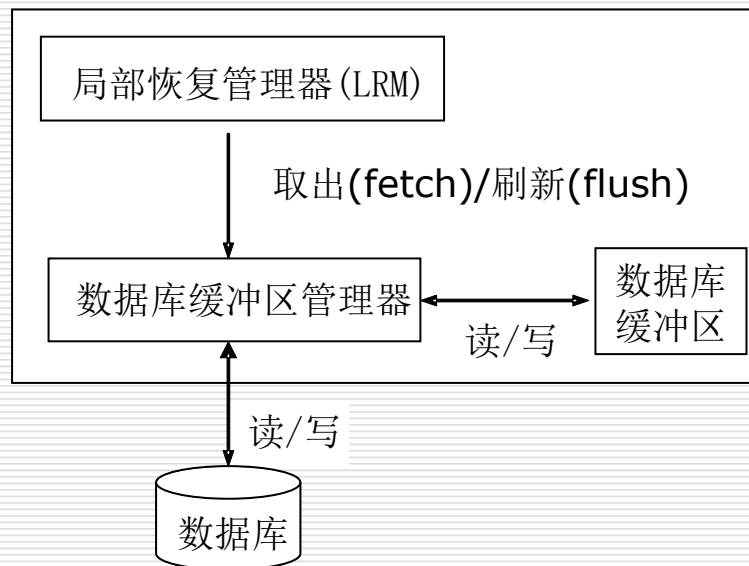


集中式数据库的故障恢复

□ 局部恢复系统的体系结构

局部恢复管理器刷新数据库缓冲区数据页的过程：

- 首先，由局部恢复管理器发出“刷新”（flush）命令并指明它想要刷新的数据页；
- 然后，缓冲区管理器在当前缓冲区中查找该数据页，若存在，则将该页数据写回到数据库中；若不存在，还要将该页数据从数据库读取到缓冲区中，将其更改后写回到数据库。



集中式数据库的故障恢复

□ 数据更新策略

(1) 数据的更新策略

数据库数据的更新通常采用两种更新方法，即原地更新和异地更新。

- ▶ **原地更新**，更新操作直接修改数据库缓冲区中的旧值。
- ▶ **异地更新**，更新操作将数据项新值存放在与旧值不同的位置上。

异地更新的两种方法：影子页面和差分文件。

影子页面：更新数据时，不改变旧存储页面，而是建立影子页面，将新值存于新建的影子页面上。旧页面用于故障恢复。

差分文件：由于更新操作等价于先删除旧值，再插入新值。因此，差分文件(F)由只读部分(FR)加上插入部分(DF^+)和删除部分(DF^-)组成，即 $F = (FR \cup DF^+) - (DF^-)$ 。

集中式数据库的故障恢复

□ 数据更新策略

(2) 缓冲区的更新策略

缓冲区更新策略有固定/非固定(fix/non_fix)和刷新/非刷新(flush/no_flush)。

- ▶ **固定/非固定(fix/non_fix)**: 事务在执行过程中, 缓冲区管理器是否要等LRM下发命令, 才将缓冲区中修改的内容写到外存数据库;
- ▶ **刷新/非刷新(flush/no_flush)**: 在事务执行结束后, LRM是否强制缓冲区管理器将当前缓冲区中已修改数据写回外存数据库;

四种缓冲区更新策略:

- ▶ **fix/flush**
- ▶ **fix/no_flush**
- ▶ **non_fix/flush**
- ▶ **non_fix/no_flush**

集中式数据库的故障恢复

□ 针对不同的缓冲区更新策略的恢复方法

(1) fix/flush

--**特点**: 事务执行过程中, 缓冲区页面固定, 废弃事务的修改不会被写入外存; 事务结束后, **LRM**强制将缓冲区中的事务修改写回到外存数据库。

--**处理过程**: ① 事务结束后, **LRM**向缓冲区管理器发**flush**命令, 将更新数据刷新到外存; ② **LRM**向缓冲区管理器发**unfix**命令释放所有被固定的页面; ③ **LRM**向日志文件中写事务结束记录。

--**恢复操作**: 不需做任何恢复操作。

集中式数据库的故障恢复

□ 针对不同的缓冲区更新策略的恢复方法

(2) fix/no_flush

--特点：事务执行过程中，缓冲区页面固定，废弃事务的修改不会被写入外存；事务结束后，LRM不强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务可能没有把修改刷新到外存数据库；

--处理过程：①LRM向日志文件中写事务结束记录；②LRM向缓冲区管理器发unfix命令释放所有被固定的页面。

--恢复操作：已提交的事务进行重做(redo)处理。

集中式数据库的故障恢复

□ 针对不同的缓冲区更新策略的恢复方法

(3) no_fix/flush

--特点：事务执行过程中，缓冲区页面不固定，废弃事务的修改可能被写入外存；事务结束后，LRM强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务一定把修改刷新到了外存数据库；

--处理过程：①LRM向缓冲区管理器发flush命令，将更新数据刷新到外存；
②LRM向日志文件中写事务结束记录。

--恢复操作：对未提交事务全部做反做(undo)处理。

集中式数据库的故障恢复

□ 针对不同的缓冲区更新策略的恢复方法

(4) no_fix/no_flush

--特点：事务执行过程中，缓冲区页面不固定，废弃事务的修改可能被写入外存；事务结束后，LRM不强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务不一定把修改刷新到了外存数据库；

--处理过程：LRM向日志文件中写事务结束记录。

--恢复操作：见下页。

集中式数据库的故障恢复

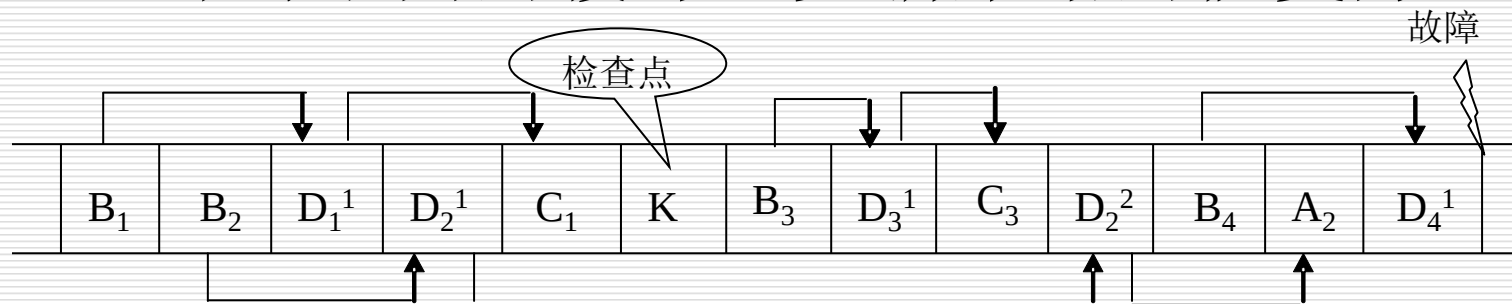
□ 针对不同的缓冲区更新策略的恢复方法

(4) no_fix/no_flush

--恢复操作：①LRM首先从重启动文件中取得最近的检查点的地址，然后建立两个表：重做表和反做表。初始化两表，重做表初态为空；反做表初态为检查点上活动事务。②确定重做事务和反做事务：在日志中从检查点向日志结尾扫描。如果发现有**新开始**（B记录）的事务，放入undo表；如果发现有**提交**（C记录）的事务，从undo表中删除，写入redo表。③反做undo表中的所有事务，重做redo表中的所有事务。

集中式数据库的故障恢复

□ 针对不同的缓冲区更新策略的恢复方法



要求：根据日志文件确定redo表和反做undo表。

步骤1：初始化redo表和undo表。在检查点上没有结束的事务T2为活动事务，放入undo表中。

步骤2：从检测点开始向扫描日志末尾扫描，如有新开始的事务，放入undo表；如有提交事务，从undo表中删除，放入redo表，直到日志文件结束。

步骤3：反做undo事务表中的事务{T2,T4}，重做redo事务表{T3}；

初始化：redo表={}, undo表={T2}

开始扫描，遇到B₃：

redo表={}, undo表={T2} ∪ {T3}

继续扫描，遇到C₃：

redo表={ } ∪ {T3}，

undo表={T2, T3} - {T3}={T2}

继续扫描，遇到B₄：

redo表= {T3}，undo表={T2} ∪ {T4}

继续扫描，遇到A₂：

redo表= {T3}，undo表={T2,T4}

分布式事务的故障恢复

在分布式数据库系统中，除了可能出现集中式数据库系统中的故障外，还可能出现分布式数据库系统特有的故障。

- 发生在各场地内部的故障称为**场地故障**。
- 发生在场地间通信过程中的故障称为**通信故障**。

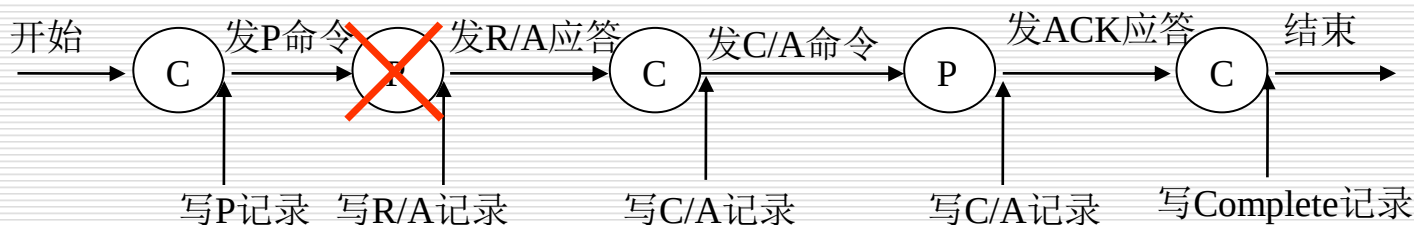
分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障

C: 协调者

P: 参与者



场地故障情况1: 在参与者场地，若参与者在写R/A记录（准备提交/放弃）之前出错。

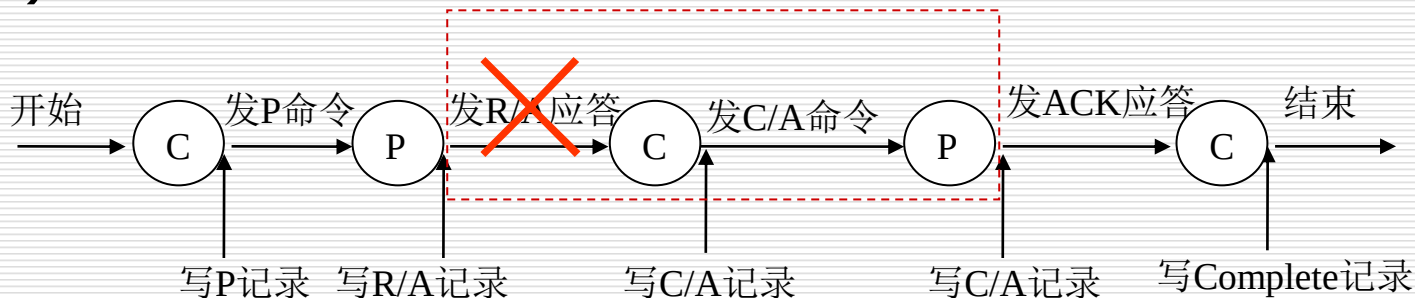
现象: 协调者发完P命令（预提交）后在规定时间内收不到应答信息。

恢复策略: 协调者做“废弃”处理，即缺省认为收到A应答。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况2：在参与者场地，若参与者在写R/A记录之后，写C/A记录之前出错。

讨论①：出错时参与者已写R/A记录，但未发送R/A应答。

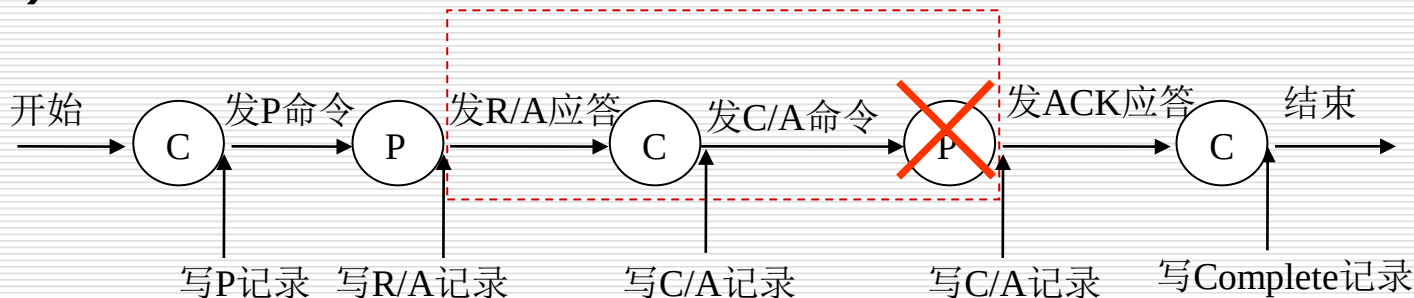
现象：由于协调者没有收集到所有参与者返回的R/A应答，因此处于等待状态。

恢复策略：故障参与者恢复后，写A记录，发送A应答。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况2： 在参与者场地，若参与者在写R/A记录之后，写C/A记录之前出错。

讨论②： 出错时参与者已经将R/A应答返回给协调者。协调者发送了C/A命令，等待参与者返回ACK应答。

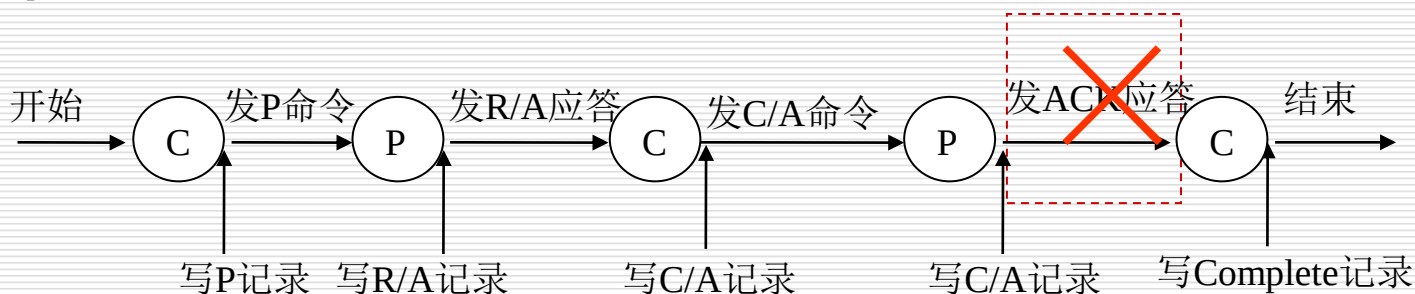
现象： 由于协调者没有收集到所有参与者返回的ACK应答，因此处于等待状态。

恢复策略： 协调者给故障参与者重发C/A命令。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况3: 在参与者场地，若参与者在写C/A记录之后出错。

讨论①: 在参与者场地，若参与者在写C/A记录之后出错，但没有返回ACK应答。

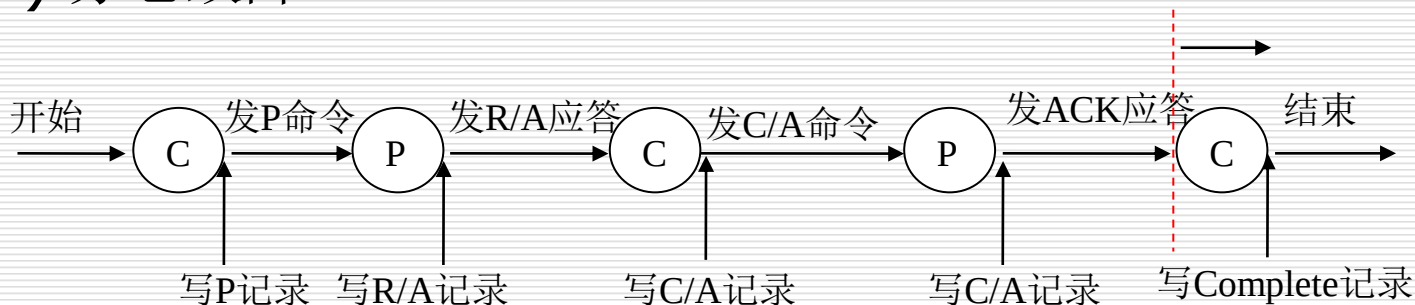
现象: 由于协调者没有收集到所有参与者返回的ACK应答，因此处于等待状态。

恢复策略: 协调者给故障参与者重发C/A命令。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况3：在参与者场地，若参与者在写C/A记录之后出错。

讨论②：出错时参与者已发送ACK应答。

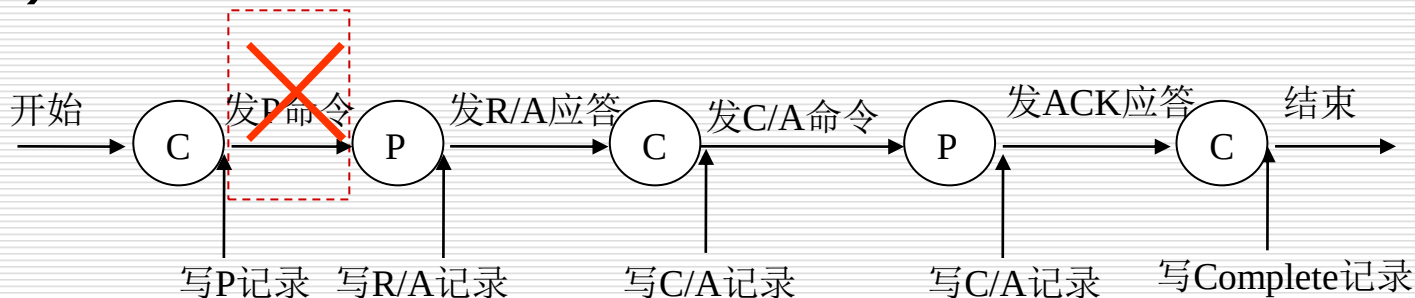
现象：协调者收集到所有参与者返回的ACK应答，事务正常结束。

恢复策略：不需要任何恢复处理。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况4: 在协调者场地，协调者在写P记录之后，写C/A记录之前出错。

讨论①: 出错时协调者未将P命令发送给参与者。

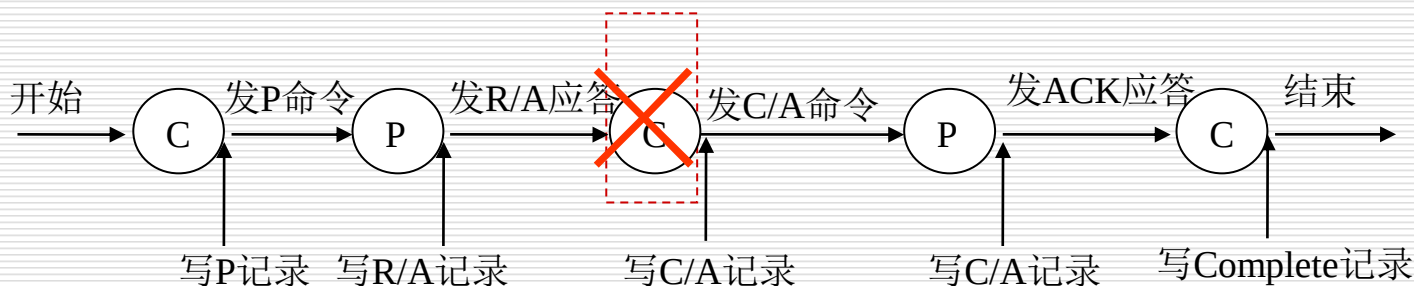
现象: 无

恢复策略: 协调者重新启动后，重新执行两段提交协议。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况4：在协调者场地，协调者在写P记录之后，写C/A记录之前出错。

讨论②：出错时协调者已将P命令发送给参与者。

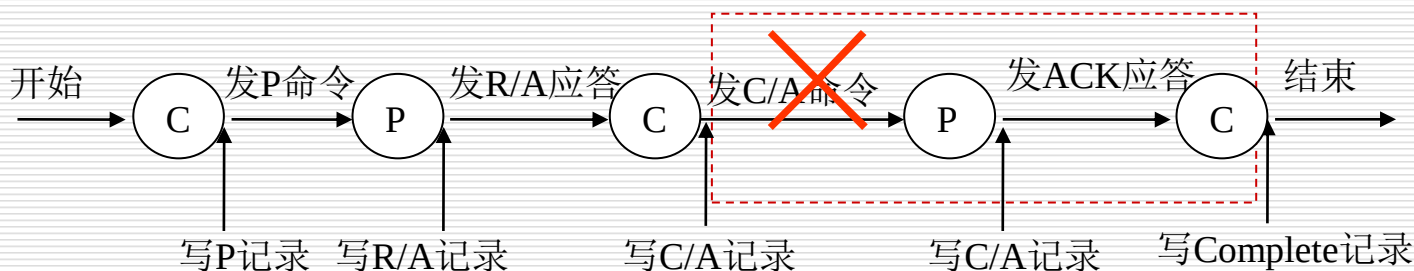
现象：此时，参与者已经发送了R/A应答，处于等待状态，等待协调者的C/A命令。

恢复策略：参与者发现超时，启动两阶段提交协议的终结协议。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况5: 在协调者场地，协调者在写C/A记录之后，写Complete记录之前出错。

讨论①: 出错时协调者未将C/A命令发送给参与者。

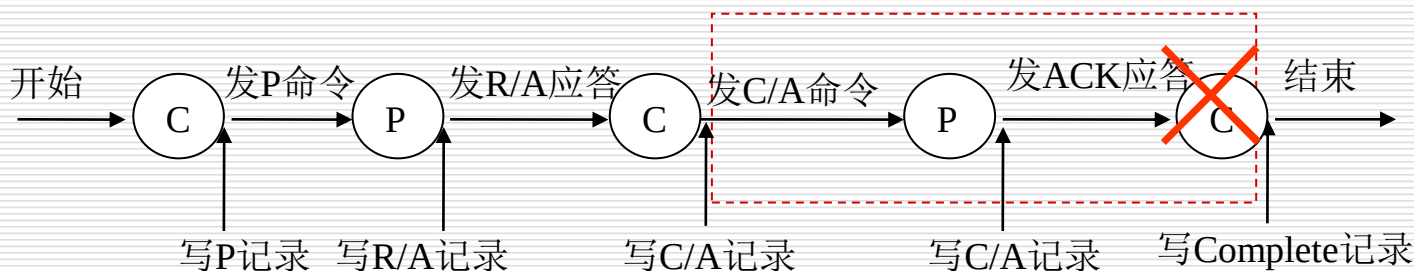
现象: 此时，参与者已经发送了R/A应答，处于等待状态，等待协调者的C/A命令。

恢复策略: 参与者发现超时，启动两阶段提交协议的终结协议。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障



场地故障情况5：在协调者场地，协调者在写C/A记录之后，写Complete记录之前出错。

讨论②：出错时协调者已将C/A命令发送给参与者。

现象：此时，协调者在故障过程中可能错过了对参与者ACK应答的接收。

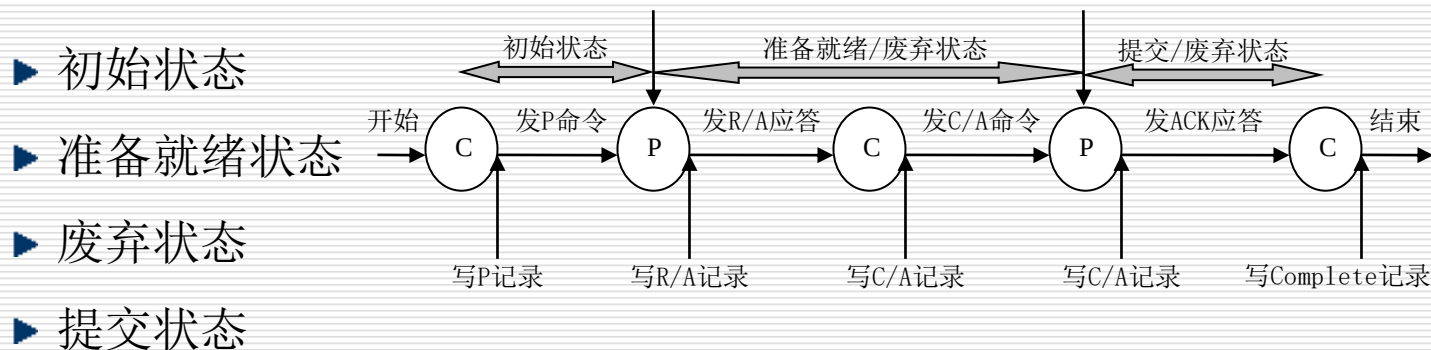
恢复策略：协调者重新启动后，给所有参与者重发其A/C命令。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障

两阶段提交协议的**终结协议**的核心思想是：**参与者**通过访问其它参与者的当前状态来推断协调者的上一个决定，从而确定终结类型（全局提交还是全局废弃）。



注意：只有当参与者在成功发送完消息后才完成状态的转换。另外，**不存在比其余参与者多于一次状态转换的参与者**，即不存在“初始”和“提交”并存的情况。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(1) 场地故障

两阶段提交协议的终结协议的内容：

步骤1：选择参与者 P_T 作为新的协调者。

步骤2： P_T 向所有参与者发送“访问状态”命令，各参与者返回自身的状态。

步骤3： P_T 根据各参与者当前的状态做出决定：

（1）若部分参与者处于“初始”状态，部分参与者处于“准备就绪”状态，则 P_T 发送Abort命令；

（2）若所有参与者均处于“准备就绪”状态，则 P_T 发送Commit命令；

（3）若至少一个参与者处于“提交”状态，则 P_T 发送Commit命令；

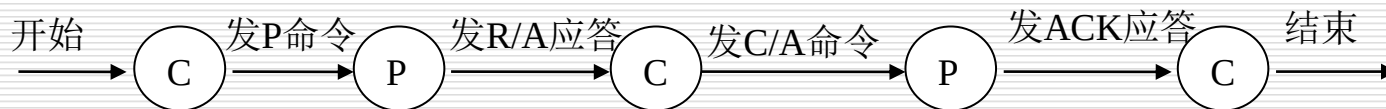
（4）若至少一个参与者处于“废弃”状态，则 P_T 发送Abort命令；

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(2) 通信故障

通信故障有两种：**报文丢失**和**网络分割**。



丢失P报文：此时，参与者未收到协调者发送的P命令，故不予应答，协调者处于等待R/A应答状态。协调者发现超时，做“废弃”处理。

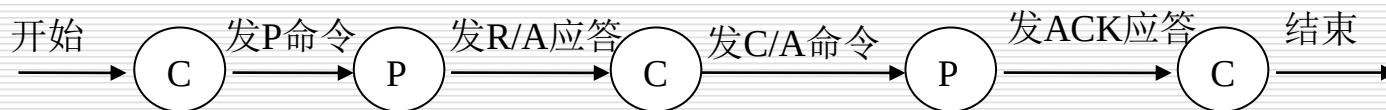
丢失R/A应答报文：此时，协调者没有收集到所有参与者返回的应答，默认做“废弃”处理。

分布式事务的故障恢复

□ 两阶段提交协议对故障的恢复

(2) 通信故障

通信故障有两种：**报文丢失**和**网络分割**。



丢失C/A命令报文：此时，参与者没有及时收到来自协调者的命令，因而处于等待状态。如果参与者发现等待超时，可以申请协调者重新发送C/A命令，或者启动两阶段提交协议的终结协议。

丢失ACK应答报文：此时，协调者没有收集到全部的ACK应答而处于等待状态。协调者如果发现超时，向参与者重发C/A命令，要求参与者给予应答。

分布式可靠性协议

□ 可靠性与可用性

可靠性 (Reliability)：在给定环境条件下和规定的时间内，数据库系统符合其行为规范的概率。**强调正确性。**

可用性 (Availability)：在给定时刻 t 上，数据库系统正常运行的概率。**强调可运行的能力。**

分布式可靠性协议

□ 可靠性与可用性

可靠性和可用性往往相互矛盾。例如，数据a的副本分别存放在n个场地上，某事务采用两段提交协议来更新a。当协调者已决定提交，但在给各个参与者发送提交命令时发生通信故障。此时：

- ▶ 若要保证系统的可靠性，则要求各个参与者保持等待而不允许其它事务使用a及其副本，以免造成数据的不一致，这就降低了系统的可用性；
- ▶ 若要保证系统的可用性，则允许其它事务使用a及其副本，但降低了系统的可靠性；

分布式可靠性协议

□ 可靠性与可用性

可靠性的度量方法：

在某时间段内系统发生失败的次数服从泊松分布，即在 $[0, t]$ 的时间段内系统发生 k 次失败的概率为：

$$\Pr\{k \text{ failures in time } [0, t]\} = \frac{e^{-m(t)}[m(t)]^k}{k!},$$

$$\text{where } m(t) = \int_0^t z(x)dx.$$

进一步地，在 $[0, t]$ 的整个时间段内系统发生0次失败的概率。

$$R(t) = \Pr\{0 \text{ failures in time } [0, t]\} = e^{-m(t)}$$

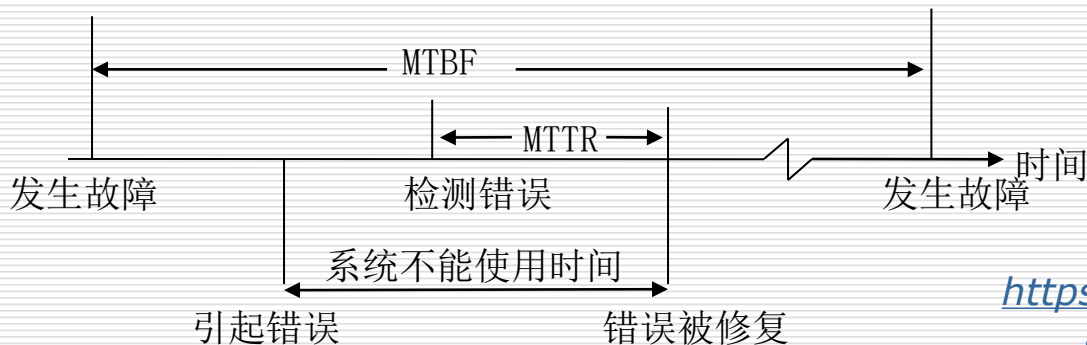
分布式可靠性协议

□ 可靠性与可用性

可用性的度量方法：

平均故障间隔时间（MTBF）：MTBF是在可以自我修复的系统中相继的两次失败之间的期望时间。

平均故障修复时间（MTTR）：MTTR是修复一个失败系统所需要的期望时间，该值与系统的修复概率相关。



$$A = \frac{MTBF}{MTBF + MTTR}$$

相关易混淆概念的解释

https://blog.csdn.net/yunhua_lee/article/details/121674703

大数据库的恢复管理

- 大数据库系统每天需要操作的数据量越来越大，集群规模也在不断扩大，难免会发生一些故障。
- **容错**就是当由于种种原因在系统中出现了数据损坏或丢失时，系统能够自动将这些损坏或丢失的数据恢复到发生事故以前的状态，使系统能够连续正常运行的一种技术。

大数据数据库的恢复管理

□ 大数据数据库的恢复管理问题

- **支持自适应的故障检测：**支持自适应的故障检测是数据库系统具有良好容错性能的前提。
- **支持与传统恢复管理策略的兼容性：**将传统分布式数据库中基于事务的故障恢复策略与**多副本管理**策略相结合。
- **支持系统的高可用性：**为了保证系统的高可用性，大数据数据库系统中的数据一般被**冗余存储**。
 - 利用冗余资源来替代故障节点继续提供服务
 - 保证故障发生后这些冗余资源之间的一致性

大数据数据库的恢复管理

- 大数据数据库系统中的故障类型：事务内部的故障，系统故障，存储介质故障，通信故障。
- 大数据数据库系统的故障检测技术
 - 基于心跳机制的故障检测
 - 基于租约机制的故障检测
 - 基于Gossip协议的故障检测
- 大数据数据库系统的容错技术
 - 基于事务的大数据库容错技术
 - 基于冗余的大数据库容错技术
 - 以强一致性为目标的容错处理方法
 - 以最终一致性为目标的容错处理方法

大数据库系统的故障检测技术

□ 基于心跳机制的故障检测

- **基本思想**：被监控的进程（或节点）**定期发送心跳信息**给监控进程（或节点），若在给定时间内监控进程（或节点）没有收到心跳信息，则认为被监控进程（或节点）失效。
- **例如**，假设**A**和**B**分别为监控进程和被监控进程，**A**需要确认**B**是否发生故障，那么**A**每隔一段时间向**B**发送一个心跳包。如果一切正常，**B**将响应**A**的心跳包，并向**A**做出回应；若**A**重试一定次数后仍未收到**B**的响应，则认为机器**B**发生了故障。
- **存在的问题**：会引发“双主”问题。如果**B**过于繁忙，都会导致**B**无法立即响应来自**A**的心跳包。若此时**A**认为**B**发生了故障，则**A**需要将**B**上面的服务迁移到集群中的其他服务器。而当网络问题被解决或者**B**闲下来后，**B**可能会继续提供服务，这样将出现多台服务器同时服务同一份数据而导致数据不一致的情况。

大数据库系统的故障检测技术

□ 基于租约机制的故障检测

- **基本思想**：监控进程定期向被监控进程**发放租约**，被监控进程持有的租约在有效期内才允许提供服务，否则主动停止服务。
- **例如**：**A**需要检测**B**是否发生故障，**A**给**B**发放租约，**B**持有的租约在有效期内才允许提供服务。正常情况下，**B**通过不断申请租约来延长有效期。租约延长请求和批准确认信息通常都是附加在**A**和**B**之间的心跳消息中来传递的。当**B**出现故障或者**B**与**A**之间的网络故障时，**B**的租约将过期而主动停止服务，从而**A**能够确保**B**不再提供服务，将**B**上面的服务迁移到其他服务器。

大数据数据库系统的故障检测技术

□ 基于租约机制的故障检测

- Google Bigtable的GFS系统中，Master作为监控方，而Chunk服务器作为被监控方。GFS系统通过租约机制将Chunk写操作授权给Chunk服务器，获取租约授权的Chunk服务器称为Primary Chunk服务器，其它副本所在的Chunk服务器称为Secondary Chunk服务器。当因故障，Primary Chunk服务器主动停止服务时，Master将产生一个加诸于其他副本的新租约。

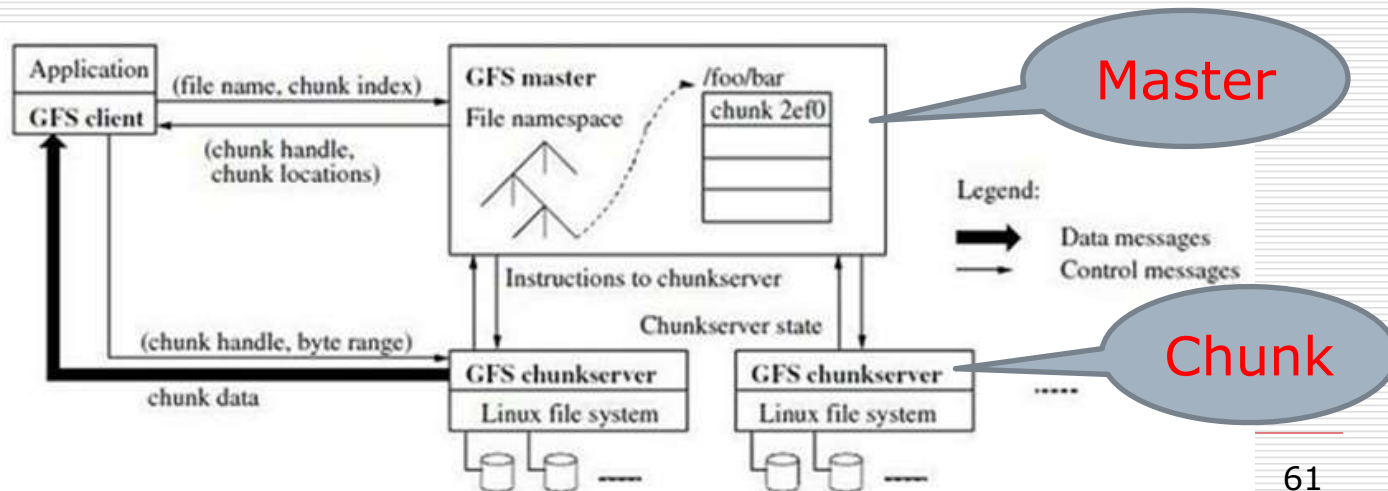


Figure 1: GFS Architecture

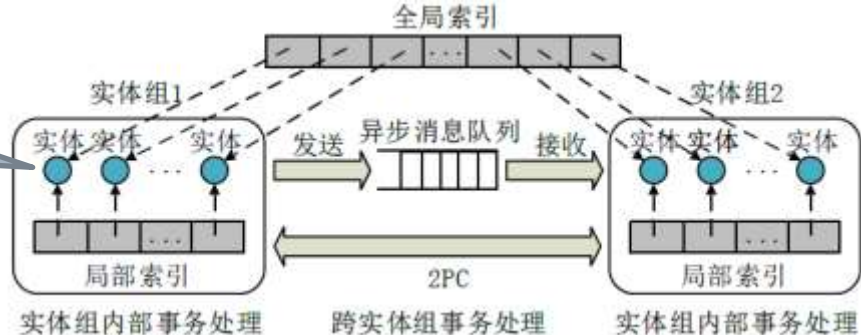
大数据数据库系统的故障检测技术

□ 基于**Gossip**协议的故障检测

- **基本思想**：**Gossip**协议又被称为反熵（Anti-Entropy）协议，就是在杂乱无章中寻求一致。**Gossip**协议的基本思想是：每个节点都随机地与其他节点通信，经过一番杂乱无章的通信，**最终**所有节点上的信息表状态都会**达成一致**。
- **Gossip**协议具有分布式容错的优点，即使有的节点因宕机而重启，或者有新节点加入，但经过一段时间后，这些节点的状态也会与其他节点达成一致。虽然**Gossip**协议无法保证在某个时刻所有节点状态一致，但可以保证在最终时刻所有节点状态一致。因此，**Gossip**协议是**最终一致性协议**。
- **Dynamo**和**Cassandra**这两种大数据数据库系统均采用了基于**Gossip**协议的故障检测技术。

实体组

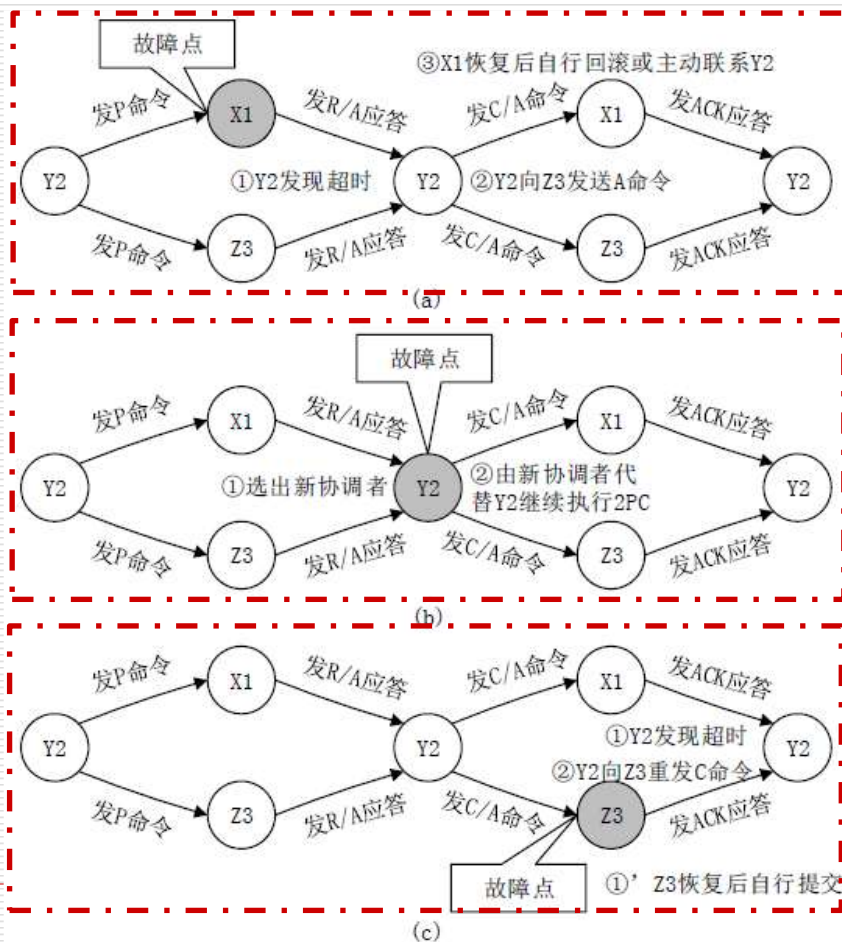
基于事务的大数据库

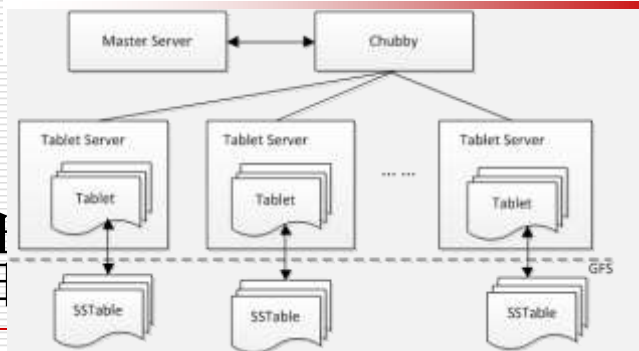


- 若大数据库系统支持事务，数据库恢复管理器主要依据数据库日志文件对数据库事务进行恢复。
- Megastore系统：
 - 同一个实体组内部支持满足**ACID特性**的事务。将同一个实体组的Redo Log记录到该组的根实体中，对应Bigtable系统中的一行，从而保证Redo Log操作的原子性。
 - Redo Log被写完后，需要对其回放，即按照Redo Log将事务中的更新操作永久地作用到数据库中。
 - 如果在写完Redo Log后、回放Redo Log前系统发生了故障（如：某些行所在的Tablet Server宕机），则回放Redo Log失败，此时事务操作仍可成功返回客户端。
 - 因为后续的读操作在读取数据时需要先回放Redo Log，这样仍能保证读取到最新的数据。因此，当Redo Log被写完后，即可认为事务操作成功。

基于事务的大数据库容错技术

Spanner系统基于2PC协议来实现分布式事务



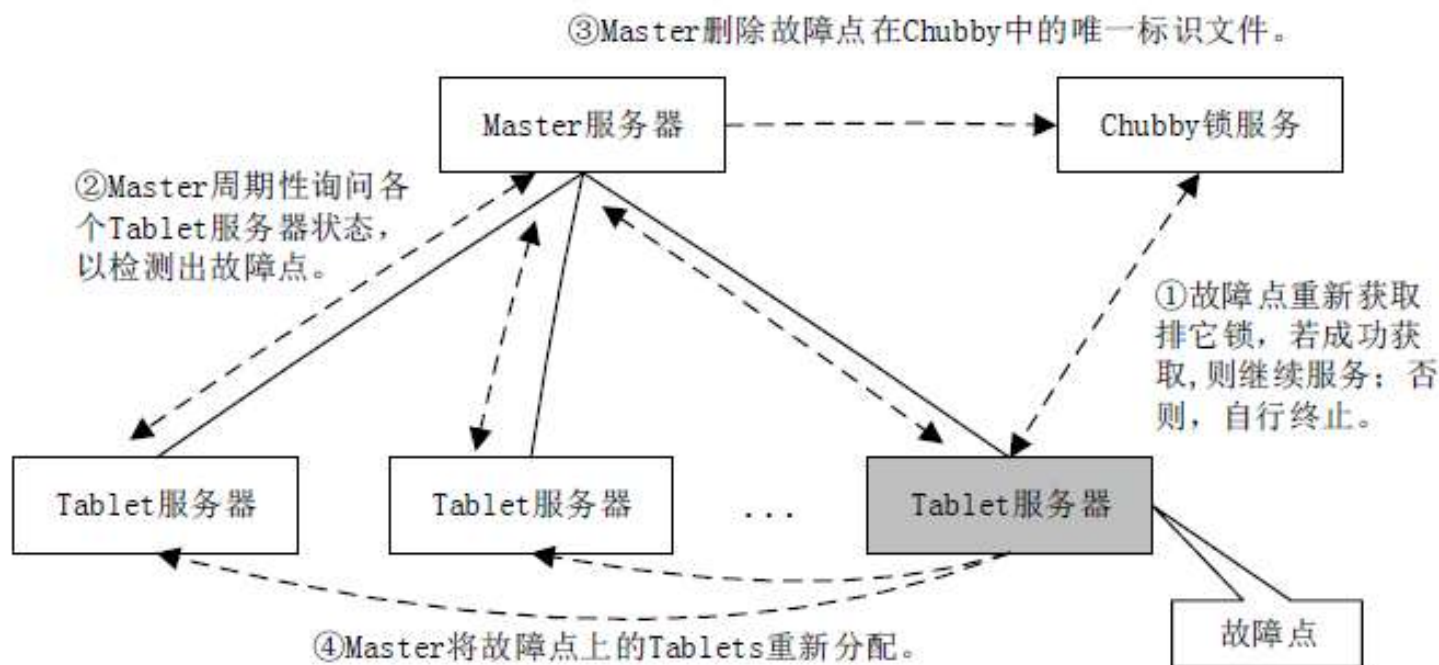


基于冗余的大数据库容错

- **基本思想：**在系统发生故障时，以降低系统性能为前提，将原有的数据和服务迁移到其他正常工作的节点上，利用冗余资源完成故障恢复。
- **以强一致性为目标的容错处理方法**
 - **以Bigtable为例：**Bigtable是基于Chubby进行容错处理的，Chubby是一个高可用的、序列化的分布式锁服务组件。一个Chubby服务包括多个活动的副本，在任何给定的时间内最多只有一个副本被选为Master。当启动一个Tablet服务器时，系统会在特定的Chubby目录下，对一个唯一标识的文件获得排他锁。Master会通过该目录检测Tablet服务器状态。Bigtable中的故障可分为Tablet服务器故障和Master故障。

基于冗余的大数据库容错技术

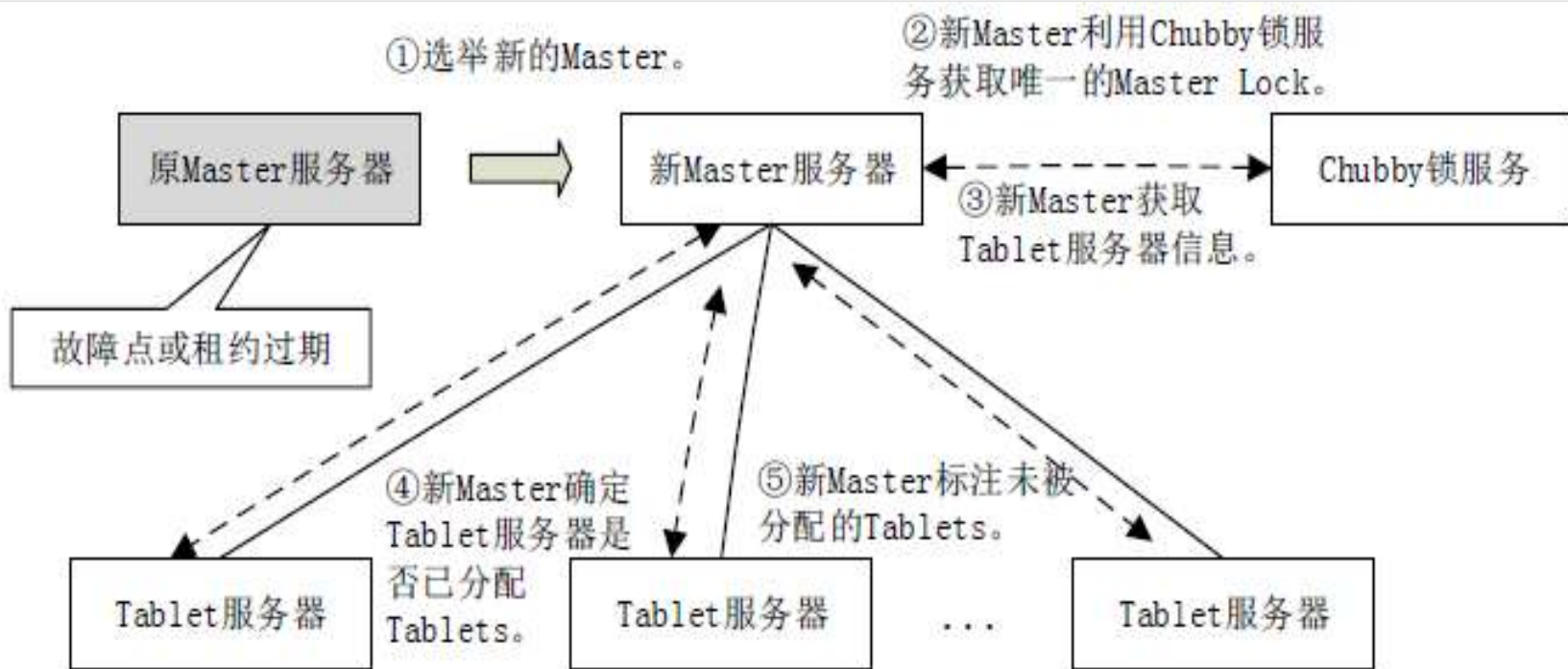
- 以强一致性为目标的容错处理方法
 - Tablet服务器故障的恢复



基于冗余的大数据库容错技术

□ 以强一致性为目标的容错处理方法

■ Master故障的恢复



基于冗余的大数据库容错技术

□ 以**最终一致性**为目标的容错处理方法

- 最终一致性允许某时间片段内副本间数据的不一致，在该时间片段（不一致窗口）之后可以保证数据的一致性。
- 以最终一致性为目标的容错处理方法通过**牺牲部分一致性**，采用**多副本冗余**的方式来保证系统的**高可用性**。
- 在大数据库系统中，如Dynamo、Cassandra以及HBase等，均采用了以最终一致性为目标的容错处理方法。

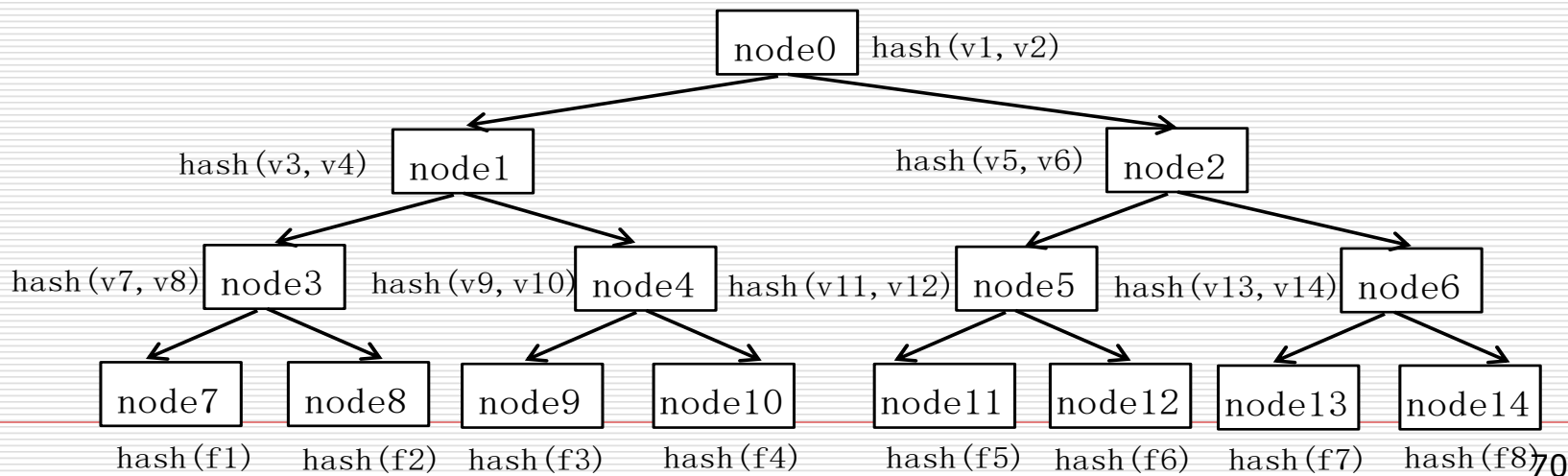
基于冗余的大数据库容错技术

□ 以最终一致性为目标的容错处理方法

- 在Dynamo中，采用多副本冗余的方式将一份数据分别写到编号为 K 、 $K+1$ 、...、 $K+N-1$ 的 N 台机器上。Dynamo的容错机制主要包括数据回传和数据同步两个阶段：
- 数据回传：若编号为 $K+i$ 的机器($0 \leq i \leq N-1$)宕机或无法连接，原本写入该机器的数据将转移到编号为 $K+N$ 的机器上。若在给定的时间 T 内机器 $K+i$ 恢复，并且机器 $K+N$ 通过Gossip机制感知到机器 $K+i$ 的恢复，则机器 $K+N$ 将数据回传到机器 $K+i$ 上。
- 数据同步：若超过时间 T ，机器 $K+N$ 仍未感知到机器 $K+i$ 恢复，则机器 $K+N$ 认为机器 $K+i$ 发生了永久性异常，机器 $K+N$ 将通过Merkle Tree机制从其他副本（ K 、...、 $K+i-1$ 、 $K+i+1$ 、...、 $K+N-1$ ）进行数据同步。同一数据的不同副本在进行数据同步时分别对数据集生成一个Merkle Tree。

基于冗余的大数据库容错技术

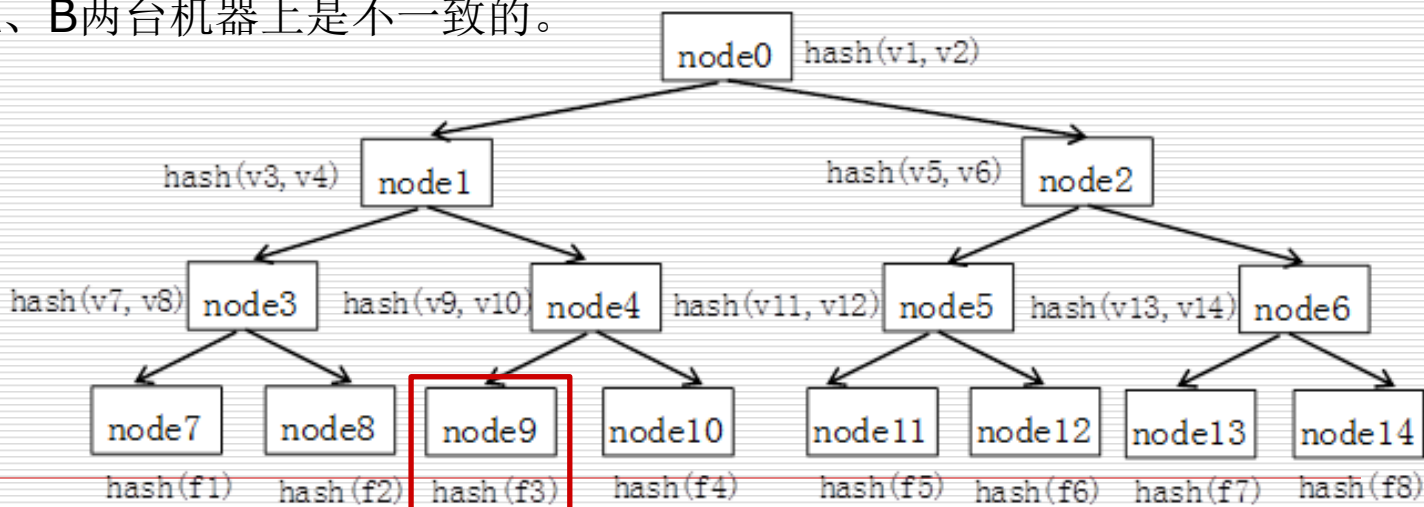
- 以最终一致性为目标的容错处理方法
- **Merkle Tree** 又称哈希树，可以是二叉树，也可以是多叉树。
Dynamo通过比较不同副本的Merkle Tree来确定副本之间是否一致，并可以在 $\log(N)$ 时间内快速的定位是哪部分发生了变化。
- **例如**，假设有A和B两台机器，分别存储了8个文件(f1~f8)的副本。为了确定A和B上所保留的副本是否一致，在文件创建时每个机器都将构建一棵Merkle Tree。



基于冗余的大数据库容错技术

□ 以**最终一致性**为目标的容错处理方法

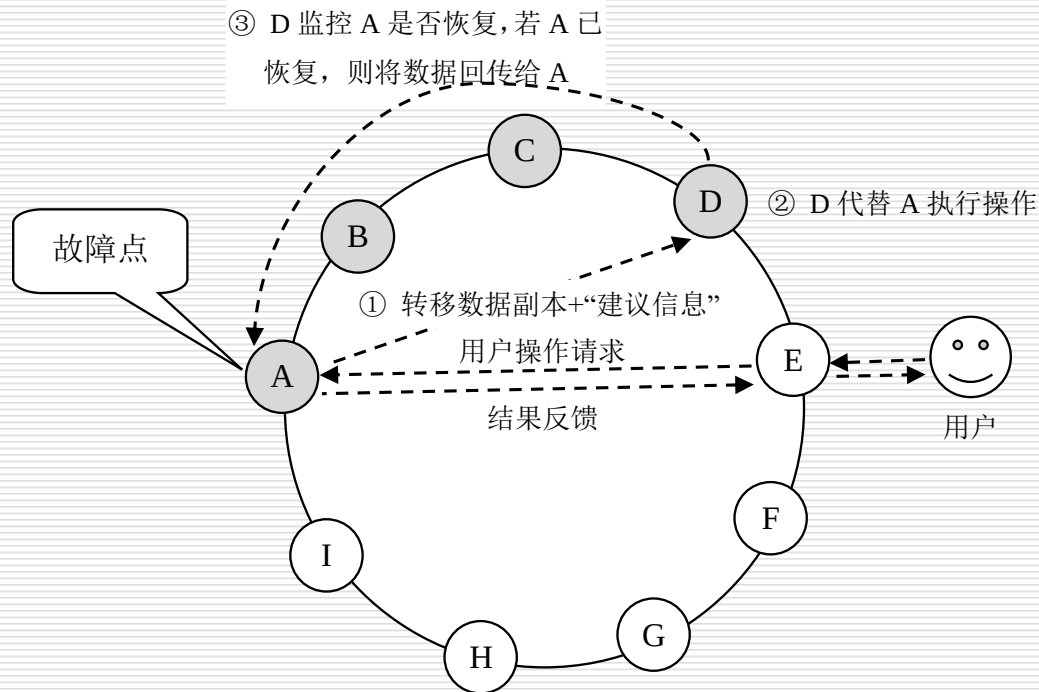
- 叶子节点保存的是数据集合的单元数据或者单元数据哈希值（如node7的值为f1的哈希值）。非叶子节点保存的是子节点值的哈希值（如node3的值是根据node7和node8的值进行计算而得到的一个哈希值）。
- 首先比较A、B上Merkle Tree中最顶层的节点，如果相等，则认为副本一致。否则，再分别比较左右子树。
- 如**f3不一致**，最终可定位到叶子节点**node9**，获取其目录信息，即可判定f3在A、B两台机器上是不一致的。



基于冗余的大数据库容错技术

□ Dynamo系统的容错处理过程

- Dynamo基于**NRW协议**来指定数据的一致性，N表示数据所具有的副本数，R和W分别表示完成读或写操作所需要读取或写入的最小副本数。
- Dynamo通过设置这三个数值来灵活地调整系统的可用性与一致性。如果 $W+R>N$ ，可保证当不超过一台机器发生故障的时候，至少能读到一份有效的数据。Dynamo推荐使用322（N、R、W的取值分别是3、2、2）的组合。





第七章 结束